

PYTHON

05

Django REST Framework (REST API Development)

Build real-world APIs that apps and companies actually use — from your first endpoint to live deployment.

DURATION

6 Sessions

LEVEL

Intermediate

MODULE

Module 5

UPDATED

May 2026

IN THIS MODULE

01

Introduction to REST & DRF Setup

02

Serializers + CRUD API Development

03

ViewSets, Routers & Pagination

04

Authentication & Permissions

05

Third-Party API Integrations

06

Advanced Features + Deployment

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to REST & DRF Setup

◆ Learning Objectives

- Explain what an API is and why REST has become the global industry standard
- Distinguish between JSON and XML as data-exchange formats and know when to use each
- Identify the key differences between a plain Django view and a DRF APIView
- Install DRF and register it correctly inside an existing Django project
- Build and test your first 'Hello API' endpoint using APIView

01 What Is an API and Why Does REST Win?

An API (Application Programming Interface) is a contract between two programs: one asks for data, the other sends it back. Zomato's mobile app does not run a database on your phone — it calls Zomato's API which replies with restaurant names, prices, and order status in milliseconds. REST (Representational State Transfer) is the set of rules that makes APIs predictable, scalable, and easy to consume from any device or language.

CONCEPT — The Four REST Principles

1. Resource-based URLs — every 'thing' gets its own URL: `/api/doctors/`, `/api/doctors/5/`.
2. Stateless — the server never remembers the previous request; every call is self-contained.
3. Standard HTTP verbs — GET (read), POST (create), PUT/PATCH (update), DELETE (remove).
4. JSON responses — structured data the client can immediately use.

SHARE OF ALL WEB APIS THAT USE REST

83%

Source: Postman State of the API Report, 2023

02 JSON vs XML — Pick Your Format

JSON (Use This)

Lightweight and human-readable. Natively parsed by JavaScript, Python, and every modern language. Used by Razorpay, Google Maps API, GitHub. Example: `{"doctor": "Dr. Shah", "city": "Ahmedabad"}`. DRF returns JSON by default — no extra configuration needed.

XML (Legacy Systems Only)

Verbose and tag-heavy. Used in older banking and government SOAP services. The same data as JSON — `<doctor><name>Dr. Shah</name><city>Ahmedabad</city></doctor>` — is twice the payload size. Avoid unless a client explicitly requires it.

03 Django View vs DRF APIView

A standard Django view is built to return an HTML page rendered from a template — perfect for web portals. But when a React frontend or an Android app calls your backend, it does not want HTML. It wants clean, structured JSON. DRF replaces Django's `HttpResponse` with a smarter `Response` class that automatically serializes Python data into JSON and handles content negotiation.

Feature	Django View	DRF APIView
Default output	HTML page	JSON response
Data validation	Django Forms	Serializer
Auth options	Session only	Token, Session, JWT, OAuth
Browsable interface	None	Built-in API browser at any endpoint URL
Best for	Web portals	Mobile apps, React/Vue frontends, microservices

04 Installing and Wiring Up DRF

```

pip install djangorestframework
# In settings.py – add to INSTALLED_APPS:
'rest_framework',

python manage.py startapp api
# In project/urls.py – include the api app:
path('api/', include('api.urls')),

```

Install DRF into your virtualenv
 Open the project settings file
 Register DRF as a Django app — include the trailing comma
 Create a dedicated app for all API endpoints
 Wire the app into the main URL config
 All API endpoints will live under `/api/`

TRY THIS LIVE

Run `'pip install djangorestframework'`, add `'rest_framework'` to `INSTALLED_APPS`, then start the server with `'python manage.py runserver'`. If it starts without errors, DRF is correctly installed. No migration is needed at this step.

05 Your First 'Hello API' Endpoint

`APIView` is DRF's most basic building block. You write a class with a `get()` or `post()` method, point a URL at it, and DRF handles content negotiation, status codes, and the browsable interface automatically. Build this in your new `'api'`

app before touching any real model data.

<code># api/views.py</code>	Create this file inside the api app folder
<code>from rest_framework.views import APIView</code>	Base class for all DRF class-based views
<code>from rest_framework.response import Response</code>	Serializes Python dicts to JSON automatically
<code>class HelloAPI(APIView):</code>	Your first API class
<code>def get(self, request):</code>	Handles HTTP GET requests to this endpoint
<code>return Response({'message': 'Hello from DRF!', 'status': 'ok'})</code>	
Returns a JSON response to the caller	
<code># api/urls.py</code>	Create this file inside the api app folder
<code>from django.urls import path</code>	Standard Django URL import
<code>from .views import HelloAPI</code>	Import the view class
<code>urlpatterns = [path('hello/', HelloAPI.as_view())]</code>	
Maps /api/hello/ to HelloAPI	

TRY THIS LIVE

Visit <http://127.0.0.1:8000/api/hello/> in your browser. DRF's built-in browsable API shows a formatted page. Then open Postman, set the method to GET, and paste the same URL — you should see `{"message": "Hello from DRF!", "status": "ok"}` as clean JSON.

CASE STUDY — Juspay, Bengaluru

Context — Juspay is an Indian payment infrastructure company processing over 50 million transactions daily for Swiggy, Flipkart, Ola, and IRCTC.

Challenge — Every client — Android app, iOS app, merchant website — needed to initiate payments and check order status without the server storing any session state between calls.

What they did — Juspay built a fully stateless REST API where every request carries its own authentication token and order ID. No session is maintained on the server side, making it simple to add new servers during peak load without sharing session state.

Result — The stateless design allowed Juspay to scale horizontally during Big Billion Day and IPL ticket launches without re-architecting the backend.

Takeaway — Stateless REST is not a textbook rule — it is why India's busiest payment systems stay up during flash-sale traffic spikes.

Source: *Juspay Engineering Blog*, juspay.io/blog (2023)

“An API is a promise between two programs. REST makes that promise simple enough to keep at scale.”

WHERE TO GO NEXT

DRF is installed and your first endpoint works. The next step is connecting it to real database data — which means learning Serializers.

Explore deeper — DRF official quickstart at <https://django-rest-framework.org/tutorial/quickstart/>

Try on your own — Add a POST method to HelloAPI that reads a 'name' key from request.data and returns `{"message": "Hello, <name>!"}`

Connect the dots — Session 02 introduces Serializers and builds a full CRUD API for the Doctor model from Module 4.

Going further — Read Roy Fielding's 2000 REST dissertation, Chapter 5, at ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm — it is shorter than expected and explains exactly why statelessness matters at scale.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python backend tutor. Explain what a REST API is and why it is stateless, in simple Hinglish. Use a Zomato food ordering example where a phone app fetches restaurant data from a server. Keep it under 150 words.

SESSION

02

60 MIN · API DEVELOPMENT

Serializers + CRUD API Development

◆ Learning Objectives

- Describe what a serializer does and why it is central to every DRF endpoint
- Choose between Serializer and ModelSerializer for a given situation
- Write a ModelSerializer for the Doctor model from Module 4
- Build GET, POST, PUT, and DELETE endpoints using APIView
- Return the correct HTTP status code (200, 201, 400, 404, 204) for every scenario
- Test all four CRUD operations in Postman

01 What Is a Serializer?

A serializer is a translator that sits between your Django model (Python objects) and the outside world (JSON). When a client sends a POST request with JSON data, the serializer validates it and converts it into a Python object your model can save. When a client requests data with GET, the serializer converts the model instance back into JSON. Think of it as a customs officer — nothing gets in or out without being checked.

CONCEPT — Serialization vs Deserialization

Serialization: Python object -> JSON (server sends data out). A Doctor instance becomes {"id": 1, "name": "Dr. Shah", "city": "Ahmedabad"}. Deserialization: JSON -> validated Python object (client sends data in). DRF's serializer runs `is_valid()` before allowing any data to touch your database — bad data is rejected with a clear error message.

02 Serializer vs ModelSerializer

Serializer

You declare every field manually — full control over what is exposed and how it is validated. Useful when the API shape does not match the model (computed fields, data from multiple models). More code to write. Use this for edge cases only.

ModelSerializer

DRF reads your model and auto-generates the fields. You simply specify which fields to include. Automatically applies validators for unique, blank, and `max_length` constraints already defined on the model. Use this 90% of the time — it is faster and stays in sync with model changes.

```
# api/serializers.py
from rest_framework import serializers
from accounts.models import Doctor
```

Create this file inside the api app
Import the serializers module
Import the Doctor model built in Module 4

```
class DoctorSerializer(serializers.ModelSerializer):
```

One class handles both read and write

```
class Meta:
```

Inner class tells DRF which model and fields to use

```
model = Doctor
```

Bind this serializer to the Doctor model

```
fields = ['id', 'name', 'specialization', 'city', 'phone']
```

Expose only these fields — id is auto-added by Django

03 Building the Doctor CRUD Endpoints

CRUD maps directly to HTTP verbs across two URL patterns: a list endpoint (`/api/doctors/`) that handles GET (all records) and POST (create one), and a detail endpoint (`/api/doctors/5/`) that handles GET (one record), PUT (full update), and DELETE. Each lives in its own class in `views.py`.

```
# api/views.py – List endpoint (handles /api/doctors/)
```

GET all + POST new

```
from rest_framework.views import APIView
```

Base class

```
from rest_framework.response import Response
```

DRF JSON response

```
from rest_framework import status
```

Named HTTP status constants

```
from accounts.models import Doctor
```

The model

```
from .serializers import DoctorSerializer
```

The serializer we just built

```
class DoctorListAPI(APIView):
```

Handles list and create

```
def get(self, request):
```

GET `/api/doctors/` — returns all doctors

```
doctors = Doctor.objects.all()
```

Fetch every Doctor row

```
s = DoctorSerializer(doctors, many=True)
```

many=True converts a queryset, not a single object

```
return Response(s.data)
```

Returns a JSON array — 200 OK by default

```
def post(self, request):
```

POST `/api/doctors/` — creates a new doctor

```
s = DoctorSerializer(data=request.data)
```

Deserialize the incoming JSON body

```
if s.is_valid():
```

Run all field and model validators

```
s.save()
```

Calls `Doctor.objects.create()` — inserts into DB

```
return Response(s.data, status=status.HTTP_201_CREATED)
```

201 = new resource successfully created

```
return Response(s.errors, status=status.HTTP_400_BAD_REQUEST)
```

400 = validation failed; errors explain which field failed and why

```
# api/views.py – Detail endpoint (add below list, handles /api/doctors/<pk>/)
```

GET one + PUT + DELETE

```
class DoctorDetailAPI(APIView):
```

Handles retrieve, update, delete for one doctor

```
def get_object(self, pk):
```

Helper to avoid repeating try/except in every method

```
try: return Doctor.objects.get(pk=pk)
```

Fetch by primary key

```

except Doctor.DoesNotExist: return None
    Return None if the ID does not exist
    def get(self, request, pk):
        doctor = self.get_object(pk)
        GET /api/doctors/5/ — one doctor
        Try to fetch the doctor
    if not doctor: return Response({'error': 'Not found'},
    status=status.HTTP_404_NOT_FOUND)
    Return 404 if ID missing
    return Response(DoctorSerializer(doctor).data)
    Return serialized JSON — 200 OK
    def put(self, request, pk):
        doctor = self.get_object(pk)
        PUT /api/doctors/5/ — full update (all fields required)
        Fetch existing record
    s = DoctorSerializer(doctor, data=request.data)
    Pass both the instance and the new data
    if s.is_valid(): s.save(); return Response(s.data)
    Save changes and return the updated record
    return Response(s.errors, status=status.HTTP_400_BAD_REQUEST)
    Validation failed
    def delete(self, request, pk):
        doctor = self.get_object(pk)
        DELETE /api/doctors/5/
        Fetch the record
    if doctor: doctor.delete(); return Response(status=status.HTTP_204_NO_CONTENT)
    204 = deleted, no body returned

```

TRY THIS LIVE

Test all four operations in Postman: 1) GET /api/doctors/ — should return a JSON array. 2) POST /api/doctors/ with body {"name": "Dr. Mehta", "specialization": "Cardiology", "city": "Surat", "phone": "9876543210"} — should return 201. 3) GET /api/doctors/1/ — should return that one doctor. 4) DELETE /api/doctors/1/ — should return 204 with no body.

04 HTTP Status Codes to Memorise

Code	Name	When to Return It
200 OK	Success	GET or PUT completed successfully
201 Created	New record created	POST created a new object
204 No Content	Done, nothing to return	DELETE succeeded — no body needed
400 Bad Request	Validation failed	serializer.is_valid() returned False

Code	Name	When to Return It
401 Unauthorized	Not authenticated	Request requires login but no credentials provided
403 Forbidden	Authenticated but blocked	User is logged in but lacks permission for this action
404 Not Found	Object missing	Queried ID does not exist in the database

PRO TIP — Never Return 200 for Errors

A very common beginner mistake is returning `Response({'error': 'Not found'})` with the default 200 status. This breaks every client that checks status codes. Always pass the correct `status=` argument. DRF's `status` module provides named constants like `status.HTTP_404_NOT_FOUND` — use them instead of magic numbers so your code is readable and correct.

05 Validation — Your API's First Line of Defence

DRF automatically runs all model-level validators (`blank`, `max_length`, `unique`) during `is_valid()`. For custom business rules — like ensuring a phone number is exactly 10 digits — you add a `validate_<fieldname>()` method directly to the serializer class. If it raises a `ValidationError`, DRF returns a structured 400 response before any database call is made.

```
# Add inside DoctorSerializer, above class Meta:
Custom field-level validator
def validate_phone(self, value):
    if not value.isdigit():
        raise serializers.ValidationError('Phone must contain only digits.')
    if len(value) != 10:
        raise serializers.ValidationError('Phone must be exactly 10 digits.')
    return value
```

Method name = `validate_` + exact field name

Phone must contain only digits

DRF sends this message in the 400 response body

Indian mobile numbers are exactly 10 digits

Client sees which field failed and exactly why

Return the cleaned value if all checks pass — required

CASE STUDY — Razorpay, Bengaluru

Context — Razorpay is India's leading payment gateway, processing transactions for over 8 million businesses including Swiggy, BookMyShow, and Dunzo.

Challenge — Merchants integrate Razorpay via REST API and frequently send malformed data — wrong currency codes, negative amounts, or missing required fields — that would corrupt transaction records if allowed into the database.

What they did — Razorpay built strict serialization and validation layers that reject invalid requests at the API boundary, returning structured JSON error messages that tell the merchant's developer exactly which field failed and why — before any database write occurs.

Result — Developer integration time dropped significantly because errors are caught instantly with clear messages, rather than propagating silently into the payment flow and causing failed transactions hours later.

Takeaway — Serializer validation is not a formality — at scale, it is what keeps bad data from reaching your database and costing real money.

Source: Razorpay Engineering Blog, engineering.razorpay.com (2022)

“Validate at the gate. The further bad data travels into your system, the more expensive it is to fix.”

WHERE TO GO NEXT

You now have a full working CRUD API. The problem: every new resource needs the same 50 lines of near-identical code. Session 03 solves this with ViewSets.

Explore deeper — DRF Serializer fields reference at <https://django-rest-framework.org/api-guide/fields/>

Try on your own — Add a `validate_name()` method that rejects any doctor name shorter than 3 characters — test it in Postman with a 1-character name and observe the 400 response

Connect the dots — Session 03 collapses your two APIView classes into a single ModelViewSet — and auto-generates all URLs using a Router.

Going further — Read about `write_only=True` and `read_only=True` field options in DRF serializers — essential for handling passwords in registration endpoints and computed fields in read-only responses.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django REST Framework tutor. Explain the difference between a Serializer and a ModelSerializer in simple Hinglish. Use a hospital doctor registration form as the example. Explain what 'deserialization' and 'is_valid()' mean in plain language. Keep it under 150 words.

SESSION

03

60 MIN · API DEVELOPMENT

ViewSets, Routers & Pagination

◆ Learning Objectives

- Explain what a ViewSet is and why it eliminates repetitive CRUD code
- Refactor a two-class APIView CRUD setup into a single ModelViewSet
- Register a ViewSet with DefaultRouter and read the URLs it auto-generates
- Compare DefaultRouter and SimpleRouter and choose the right one
- Apply PageNumberPagination, LimitOffsetPagination, and CursorPagination to a list endpoint
- Add SearchFilter and OrderingFilter to a ViewSet using query parameters

01 Why ViewSets Exist

In Session 02 you wrote two classes — DoctorListAPI and DoctorDetailAPI — plus two manual URL entries to build one CRUD resource. Now imagine doing the same for patients, appointments, specializations, and clinics. Each new resource adds ~50 lines of near-identical code. ViewSets solve this by bundling all five standard actions (list, create, retrieve, update, destroy) into a single class. Add a Router, and the URLs are generated for you too — no manual urlpatterns needed.

CONCEPT — ViewSet Action Mapping

A ModelViewSet automatically provides five methods: list() -> GET /api/doctors/, create() -> POST /api/doctors/, retrieve() -> GET /api/doctors/5/, update() -> PUT /api/doctors/5/, destroy() -> DELETE /api/doctors/5/. You do not write any of these — ModelViewSet inherits them from DRF's generic mixins. You only override them when you need custom behaviour.

api/views.py – Session 03 version (replaces both Session 02 classes)

The entire Doctor CRUD in one class

```
from rest_framework import viewsets
from accounts.models import Doctor
from .serializers import DoctorSerializer
class DoctorViewSet(viewsets.ModelViewSet):
    queryset = Doctor.objects.all()
    serializer_class = DoctorSerializer
```

Import the viewsets module

Same model as before

Same serializer as before — nothing changes there

One class replaces DoctorListAPI + DoctorDetailAPI

Default queryset used by list and retrieve

Serializer used for all five actions

VIEW CODE: SESSION 02 VS SESSION 03

50 -> 4

Lines of code for the same full CRUD functionality. ModelViewSet inherits everything else.

02 Routers — URLs on Autopilot

In Session 02 you manually wrote two URL patterns for DoctorListAPI and DoctorDetailAPI. When you use a ViewSet, you hand it to a Router instead. The Router inspects the ViewSet, determines which actions are available, and generates all the correct URL patterns automatically — list, detail, and the right HTTP method bindings for each.

```
# api/urls.py – Session 03 version (replaces the manual urlpatterns)
Router does the URL work
from rest_framework.routers import DefaultRouter
    Import the router class
from .views import DoctorViewSet
router = DefaultRouter()
router.register(r'doctors', DoctorViewSet)
urlpatterns = router.urls
```

Import the new ViewSet
Create a router instance
Register the ViewSet — 'doctors' becomes the URL prefix
All generated URL patterns are in router.urls — no manual entries

Router Type	Generates API Root?	URL Suffix Support	When to Use
DefaultRouter	Yes — at /api/	Yes (.json suffix)	Most projects — gives a clickable API root in the browser
SimpleRouter	No	No	When you want clean minimal URLs with no extra endpoints
Manual urlpatterns	No	Full control	Only for non-standard ViewSet actions or mixed views

TRY THIS LIVE

After switching to the router, start your server and visit <http://127.0.0.1:8000/api/> in the browser. DefaultRouter shows a clickable API root listing all registered endpoints. Click on the doctors link — DRF's browsable interface lets you GET, POST, and test directly without opening Postman.

03 Pagination — Never Send 10,000 Records at Once

A list endpoint without pagination is a performance time bomb. If Doctor.objects.all() returns 5,000 records, every GET /api/doctors/ call serializes and transmits all 5,000 rows — crashing mobile apps on slow 4G connections and overloading the database. Pagination breaks the response into pages. DRF ships three styles, each suited to a different use case.

CONCEPT — Three DRF Pagination Styles

PageNumberPagination: client requests `/api/doctors/?page=2` — simple and human-readable, best for web UIs. LimitOffsetPagination: client controls page size and starting position via `/api/doctors/?limit=10&offset=20` — best for data exports and flexible clients. CursorPagination: uses an opaque cursor token instead of page numbers — most efficient for large, frequently-changing datasets like social feeds or live event logs.

Option 1: Global pagination for all endpoints (settings.py)

Applies to every ViewSet in the project

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
}
```

DRF global settings dictionary — add this if not already present

Use page numbers by default

10 records per page — adjust to fit your data

Close the REST_FRAMEWORK dict

Option 2: Custom pagination class per ViewSet (api/pagination.py)

Fine-grained control for one endpoint

```
from rest_framework.pagination import PageNumberPagination
```

Import the base class

```
class DoctorPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 50
```

Custom class for the Doctor endpoint only

Default records per page

Client can override: `/api/doctors/?page_size=5`

Client cannot request more than 50 records per page

In DoctorViewSet, add one line:

```
pagination_class = DoctorPagination
```

Attach the custom class to this ViewSet

DRF uses this for the Doctor list endpoint only

Style	Query Params	Best For	Key Limitation
PageNumber	?page=2	Web UIs, simple APIs	Can skip rows if data is inserted between page requests
LimitOffset	?limit=10&offset=20	Flexible clients, data exports	Slow on large offsets — DB scans from row 1 every time
Cursor	?cursor=<token>	Social feeds, live data, large tables	Cannot jump to an arbitrary page — forward-only navigation

04 Ordering & Search Filters

DRF's filter backends let clients sort and search results using query parameters — no custom queryset logic required. You declare which fields are searchable and orderable directly on the ViewSet, and DRF generates the SQL automatically. This turns a basic list endpoint into a usable search interface with two lines of code.

api/views.py – updated DoctorViewSet with filters

Add all four lines to the existing ViewSet

<code>from rest_framework import viewsets, filters</code>	Import filters alongside viewsets
<code>class DoctorViewSet(viewsets.ModelViewSet):</code>	Updated ViewSet
<code>queryset = Doctor.objects.all()</code>	Base queryset
<code>serializer_class = DoctorSerializer</code>	Serializer
<code>pagination_class = DoctorPagination</code>	Custom pagination from above
<code>filter_backends = [filters.SearchFilter, filters.OrderingFilter]</code>	
Enable both filter types on this ViewSet	
<code>search_fields = ['name', 'specialization', 'city']</code>	
GET /api/doctors/?search=Cardiology	
<code>ordering_fields = ['name', 'specialization']</code>	
GET /api/doctors/?ordering=name or ?ordering=-name (descending)	
<code>ordering = ['name']</code>	Default sort order when the client does not specify

TRY THIS LIVE

Try these in Postman or your browser after adding the filters: /api/doctors/?search=Ahmedabad (doctors in Ahmedabad only), /api/doctors/?ordering=-name (sorted Z to A), /api/doctors/?page=2&page_size=5 (page 2 with 5 records), /api/doctors/?search=Cardiology&ordering=name (cardiologists sorted A to Z).

05 When to Use ViewSet vs APIView

PRO TIP — Choosing the Right Base Class

Use ModelViewSet when your endpoint maps cleanly to a single model with standard CRUD (doctors, patients, appointments). Use APIView when your endpoint does something custom — aggregating data from multiple models, calling an external API, or performing a calculation. You can mix both in the same project. The router only works with ViewSets; APIViews still need manual URL entries.

CASE STUDY — Zomato, Gurugram

Context — Zomato serves restaurant discovery data to over 32 million monthly active users across 24 countries, with Ahmedabad and Surat among its largest Indian markets.

Challenge — A single city like Ahmedabad has 3,000+ listed restaurants. Returning all of them in one API response would crash mobile apps on 4G connections and push server response times to several seconds during peak lunch hours.

What they did — Zomato implemented cursor-based pagination on their restaurant listing API, serving 20 restaurants per response with a cursor token for the next batch. Search and ordering filters let users narrow by cuisine, rating, or delivery time before the data even leaves the server.

Result — API response times stayed under 200ms even at peak lunch hours (1–2 PM), when millions of

users simultaneously browse and order through the app.

Takeaway — Pagination and filters are not optional optimizations — they are what keep high-traffic list APIs fast and usable on real Indian mobile networks.

Source: Zomato Engineering Blog, blog.zomato.com (2022)

“Write the ViewSet once. Let the Router and DRF handle everything else.”

WHERE TO GO NEXT

Your API now lists, creates, updates, deletes, paginates, searches, and sorts — but any anonymous user on the internet can call it. Session 04 locks it down with Authentication and Permissions.

Explore deeper — DRF ViewSets and Routers guide at <https://django-rest-framework.org/api-guide/viewsets/>

Try on your own — Add a second ViewSet for the Appointment model from Module 4 — register it on the same router and confirm all five CRUD URLs are auto-generated without writing a single urlpattern manually

Connect the dots — Session 04 adds Token Authentication so only logged-in users can create or delete doctor records, and Role-based Permissions so patients cannot access admin endpoints.

Going further — Install django-filter (pip install django-filter) for field-level filtering — filter doctors by exact city name, specialization, or multiple conditions using Q() objects, beyond what DRF's built-in SearchFilter provides.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django REST Framework tutor. Explain in simple Hinglish the difference between `PageNumberPagination`, `LimitOffsetPagination`, and `CursorPagination`. Use a hospital app example where a mobile client fetches a list of doctors. Tell me which pagination style works best for an infinite-scroll mobile app and why. Keep it under 150 words.

SESSION

04

60 MIN · SECURITY

Authentication & Permissions

◆ Learning Objectives

- Distinguish between authentication (who are you?) and authorization (what are you allowed to do?)
- Compare BasicAuthentication, SessionAuthentication, and TokenAuthentication and choose correctly
- Install and configure DRF Token Authentication and generate tokens for users
- Apply AllowAny, IsAuthenticated, and IsAdminUser permission classes to a ViewSet
- Override get_permissions() to allow read-only access publicly while protecting write actions
- Write a custom permission class and describe how JWT differs from token auth

01 Authentication vs Authorization — Two Different Questions

Authentication asks 'who are you?' — it verifies the caller's identity. Authorization asks 'what are you allowed to do?' — it checks whether that identity has permission for the requested action. Think of a hospital: the front desk staff (authenticated as an employee) can view appointment slots, but only a doctor (authorized with the right role) can write prescriptions. DRF keeps these separate on purpose: authentication classes identify the caller; permission classes decide what that caller can do.

CONCEPT — DRF Request Lifecycle with Auth

When a request arrives: 1. DRF runs authentication — finds the user identity from the token or session. 2. DRF runs permissions — checks if that identity can perform this specific action. 3. If both pass, the view method executes. If authentication fails -> 401 Unauthorized. If authentication passes but permission fails -> 403 Forbidden.

02 DRF's Three Built-in Auth Backends

Auth Class	How It Works	Best For
BasicAuthentication	Sends username:password Base64-encoded in every request header	Development and quick testing only — never in production without HTTPS
SessionAuthentication	Uses Django's session cookie — same mechanism as the web portal login	Browser-based clients that share the same Django project
TokenAuthentication	Client sends a token string in the Authorization header — token is stored in DB	Mobile apps, React/Vue frontends, Postman — use this for all new projects

03 Token Authentication — Setup and Usage

```
# Step 1: Add authtoken to INSTALLED_APPS in settings.py
    DRF's token app — creates the authtoken_token table
'rest_framework.authtoken',          Add this line alongside 'rest_framework'
python manage.py migrate             Creates the token table in the database
# Step 2: Set Token Auth as the default in settings.py
    Apply globally to all endpoints
REST_FRAMEWORK = {                  DRF settings dict (create or update in settings.py)
'DEFAULT_AUTHENTICATION_CLASSES':
['rest_framework.authentication.TokenAuthentication'],
    Use tokens for all auth by default
'DEFAULT_PERMISSION_CLASSES': ['rest_framework.permissions.IsAuthenticated'],
    Require login for all endpoints by default
}
```

Close the dict

```
# Step 3: Add the built-in login endpoint to urls.py
    POST username+password here to receive a token
from rest_framework.authtoken.views import obtain_auth_token
    DRF's ready-made login view
path('api/token/', obtain_auth_token),    POST to /api/token/ with username and password —
                                         returns {"token": "abc123..."}
# Step 4: Use the token in Postman or any client
    Add to every protected request
# Header key:    Authorization          Exact header name — case-sensitive
# Header value: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4f
    Prefix 'Token ' (with a space) then the token string
```

TRY THIS LIVE

In Postman: 1) POST to /api/token/ with Body -> form-data: username=admin, password=yourpassword. You should receive a JSON token. 2) Copy the token value. 3) Make a GET request to /api/doctors/ — add Header: Authorization: Token <your_token>. You should now see the doctor list. Remove the header and you should get 401.

04 Permission Classes — Locking Down Endpoints

Permission classes run after authentication. DRF ships three you will use constantly: AllowAny lets anyone call the endpoint, even anonymous users. IsAuthenticated requires a valid login. IsAdminUser restricts to Django superusers. The real power is overriding get_permissions() on a ViewSet to apply different rules per action — public read, authenticated write.

```
from rest_framework.permissions import IsAuthenticated, AllowAny, IsAdminUser
```

Import the three main permission classes

```
class DoctorViewSet(viewsets.ModelViewSet):
    queryset = Doctor.objects.all()
    serializer_class = DoctorSerializer
    def get_permissions(self):
if self.action in ['list', 'retrieve']:
    Safe read-only actions
        return [AllowAny()]
    if self.action == 'destroy':
        return [IsAdminUser()]
    return [IsAuthenticated()]
```

Updated ViewSet
Base queryset
Serializer
Override to apply different rules per action
Anyone can view the doctor list — no login needed
Delete is restricted to admins
Only Django superusers can delete
Create and update require login — no more

PRO TIP — Public Read, Protected Write

The `get_permissions()` pattern is the most practical permission setup for most APIs — let anyone browse (list, retrieve) but require authentication to create, update, or delete. This is exactly how public APIs like GitHub work: you can view public repos without a token, but pushing code requires one.

05 Custom Permissions + JWT Overview

```
# api/permissions.py — create this file
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.created_by == request.user

# Add to DoctorViewSet:
permission_classes = [IsAuthenticated, IsOwnerOrReadOnly]
```

Custom permission for object-level access control
Base class and the safe HTTP methods tuple
Only the record's creator can modify it
Called for retrieve, update, destroy actions
SAFE_METHODS = ('GET', 'HEAD', 'OPTIONS')
Any authenticated user can view
Only the creator can edit or delete — obj must have a `created_by` field
Use the custom permission
Both conditions must pass

DRF Token Auth

Token is stored in the `authtoken_token` database table. Server does a DB lookup on every request. Easy to revoke — just delete the token row. Simple to implement with zero extra packages. Best for: small to medium projects with < 50K daily requests.

JWT (JSON Web Token)

Token is self-contained — payload + cryptographic signature, no DB lookup needed. Much faster at scale. Cannot be individually revoked without a blacklist table. Requires `django-rest-framework-simplejwt`. Best for: high-traffic APIs, mobile apps, microservices.

CASE STUDY — Zerodha Kite Connect, Bengaluru

Context — Zerodha is India's largest stock broker by active clients. Kite Connect is their REST API platform used by thousands of algorithmic traders and fintech developers.

Challenge — Financial APIs are high-value targets — a single authentication flaw could allow unauthorized stock trades on a client's behalf, causing immediate financial and legal consequences.

What they did — Zerodha implemented a two-layer token system: an API key identifies the application, and a short-lived access token (generated each trading day via an OAuth flow) authorizes specific actions. Every API call requires both. Access tokens expire each night — even a stolen token is worthless by morning.

Result — Millions of automated trades per day execute securely. Token expiry limits the blast radius of any credential compromise to a single trading day, and API key + token combination makes replay attacks impractical.

Takeaway — Token expiry and scoped permissions are not bureaucracy — they are what make financial APIs safe for production at scale.

Source: Zerodha Kite Connect API documentation, kite.trade/docs (2023)

“Authentication tells you who knocked on the door. Permissions decide whether to let them in — and into which room.”

WHERE TO GO NEXT

Your API is now locked behind tokens and permissions. Session 05 keeps the server busy in a new way — calling other APIs from inside your DRF view.

Explore deeper — DRF Authentication guide at <https://django-rest-framework.org/api-guide/authentication/>

Try on your own — Create a second test user (not admin) in the Django shell, generate their token, and verify they can GET doctors but get 403 when they try to DELETE one

Connect the dots — Session 05 introduces third-party API integrations — your DRF view will call OpenWeatherMap and Google Maps as external services.

Going further — Install `django-rest-framework-simplejwt` (pip install `django-rest-framework-simplejwt`) and replace token auth with JWT — observe the difference in response time and what disappears from the `authtoken` table.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a security-focused backend tutor. Explain in simple Hinglish the difference between Token Authentication and JWT in Django REST Framework. Use a stock trading app example like Zerodha. Explain what happens if a token gets stolen — and why JWT expiry helps. Keep it under 150 words. This is for an interview prep context.

SESSION
05

60 MIN · INTEGRATIONS

Third-Party API Integrations

◆ Learning Objectives

- Explain why a backend server should call external APIs instead of the frontend doing it directly
- Use the requests library to call an external REST API from inside a DRF view
- Build a `/api/weather/<city>/` endpoint that fetches live data from OpenWeatherMap
- Integrate the Google Maps Geocoding API to convert a city name into coordinates
- Handle external API failures — timeouts, 404s, and service outages — without crashing your endpoint
- Store API keys securely using environment variables instead of hardcoding them

01 The DRF-as-Proxy Pattern

Your DRF server can call another API, transform the response, and relay it to your client. This is called a proxy or aggregation pattern. Instead of a mobile app calling OpenWeatherMap directly, it calls your `/api/weather/` endpoint — which calls OpenWeatherMap in the background, extracts only the fields your app needs, and returns clean JSON. The client never sees raw third-party responses or secret API keys.

CONCEPT — Why Backend Should Call External APIs

Three reasons to proxy through your DRF backend: 1. Security — your API key never reaches the client's browser or app, where it could be extracted. 2. Control — you transform, filter, and cache the external response before sending it on. 3. CORS — many external APIs do not allow browser-to-API calls directly; going through your server bypasses this restriction entirely.

PUBLIC APIS AVAILABLE TO INTEGRATE TODAY

24,000+

Source: *Public APIs GitHub repository (github.com/public-apis/public-apis, 2024)*

02 The requests Library — Your HTTP Client

Django's built-in HTTP tools are for receiving requests, not making them. The requests library is the standard Python tool for calling external APIs — it handles headers, query parameters, JSON parsing, and timeouts cleanly. Install it once; use it in every integration view you build.

```
pip install requests
import requests
```

Install the requests library (add to requirements.txt too)
Import in any view file that calls an external API

```
# Basic GET call pattern:
```

Use this structure as a template for every external API call

```
response = requests.get(url, params={'key': 'value'}, timeout=5)
```

timeout=5 means give up after 5 seconds — always set this

```
response.status_code
```

HTTP status of the external API's response (200, 404, 500 etc.)

```
response.json()
```

Parse JSON body into a Python dict — no need to call json.loads() manually

03 OpenWeatherMap — Your First Live Integration

OpenWeatherMap's free tier gives 1,000 API calls per day — more than enough for a student project. Sign up at openweathermap.org, verify your email, and your API key arrives within 10 minutes. The `/data/2.5/weather` endpoint returns temperature, humidity, and weather description for any city worldwide, including every city in Gujarat.

```
# api/views.py — WeatherAPI
import requests
import os
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import AllowAny

Weather is public — no login needed
WEATHER_KEY = os.environ.get('OPENWEATHER_API_KEY', '')
Read from environment — never hardcode keys

class WeatherAPI(APIView):
    permission_classes = [AllowAny]
    def get(self, request, city):
        url = 'https://api.openweathermap.org/data/2.5/weather'
        OpenWeatherMap current weather endpoint
        params = {'q': city, 'appid': WEATHER_KEY, 'units': 'metric'}
        units=metric gives Celsius — important for Indian users
        try:
            resp = requests.get(url, params=params, timeout=5)
            5-second timeout
            if resp.status_code == 200:
                data = resp.json()
                External API succeeded
                Parse the response
            return Response({'city': data['name'], 'temp_c': data['main']['temp'], 'humidity':
                data['main']['humidity'], 'description': data['weather'][0]['description']})
            Return only the fields your client needs
            return Response({'error': 'City not found'}, status=status.HTTP_404_NOT_FOUND)
            OpenWeatherMap returns 404 for unknown cities
        except requests.exceptions.Timeout:
            External API took too long
```

```
return Response({'error': 'Weather service timed out'},
status=status.HTTP_504_GATEWAY_TIMEOUT)

504 = upstream server too slow
```

```
# api/urls.py – add this URL pattern
from .views import WeatherAPI
path('weather/<str:city>/', WeatherAPI.as_view()),

Wire the WeatherAPI view
Import alongside DoctorViewSet

Add to urlpatterns (outside the router.urls)
```

TRY THIS LIVE

Set your OpenWeatherMap key in your terminal: `export OPENWEATHER_API_KEY=your_key_here`. Then GET `/api/weather/Ahmedabad/` — you should see live temperature and humidity. Try `/api/weather/Surat/` and `/api/weather/Mumbai/`. Then try `/api/weather/FakeCity/` — confirm you get a 404 with the 'City not found' message.

04 Google Maps Geocoding API

The Google Maps Geocoding API converts a human-readable address or city name into latitude and longitude coordinates. This is what apps like Practo and Ola use to plot clinic and driver locations on a map. You need a Google Cloud account — the Geocoding API has a free tier of 28,500 calls per month.

```
# api/views.py – GeocodeAPI (add below WeatherAPI)
Converts a city name to lat/lng
GOOGLE_MAPS_KEY = os.environ.get('GOOGLE_MAPS_API_KEY', '')
Read key from environment

class GeocodeAPI(APIView):
    permission_classes = [AllowAny]
    def get(self, request):
        address = request.query_params.get('address', '')
        GET /api/geocode/?address=Satellite+Ahmedabad
        if not address:
            Guard against empty address
        return Response({'error': 'address parameter is required'},
status=status.HTTP_400_BAD_REQUEST)
        Clear 400 error if address missing
url = 'https://maps.googleapis.com/maps/api/geocode/json'
        Google Maps Geocoding endpoint
resp = requests.get(url, params={'address': address, 'key': GOOGLE_MAPS_KEY},
timeout=5)
        Call with address and API key
```

```

data = resp.json()
if data['status'] == 'OK':
    loc = data['results'][0]['geometry']['location']
    return Response({'lat': loc['lat'], 'lng': loc['lng'], 'formatted_address':
data['results'][0]['formatted_address']})
return Response({'error': 'Address not found'}, status=status.HTTP_404_NOT_FOUND)

```

Parse the response
Google returns 'OK' in the status field on success
First result's lat/lng
Return coordinates and Google's cleaned address string
Google returns ZERO_RESULTS or REQUEST_DENIED on failure

PRO TIP — Never Hardcode API Keys

Hardcoding API keys in views.py and pushing to GitHub is one of the most common and costly developer mistakes. GitHub automatically scans public repos for leaked keys — and so do bots. Always read keys using `os.environ.get('KEY_NAME')`. For PythonAnywhere or AWS, set them in the server's environment variables panel. Add a `.env` file locally and load it with `python-dotenv`.

05 Handling External API Failures

External APIs will fail — servers go down, keys expire, rate limits are hit, and network latency spikes. Your DRF endpoint must handle these cases gracefully so a failure in one third-party service does not return an unhandled Python exception (500 Internal Server Error) to your client.

Failure Type	What Happens	How to Handle It
Timeout	External server takes too long to respond	<code>timeout=</code> parameter + <code>except requests.exceptions.Timeout -> return 504</code>
Connection Error	No network or DNS failure	<code>except requests.exceptions.ConnectionError -> return 503</code>
404 from External API	Resource not found on external server	Check <code>resp.status_code == 404</code> -> return your own 404 with a clear message
429 Rate Limited	You have exceeded the API's call limit	Check <code>resp.status_code == 429</code> -> return 429 with a 'try again later' message
500 from External	External server crashed	Check <code>resp.status_code >= 500</code> -> return 502 Bad Gateway to your client

CASE STUDY — Ola Cabs, Bengaluru

Context — Ola operates ride-hailing services in 250+ Indian cities and integrates multiple external APIs — Google Maps for routing, weather APIs for demand forecasting, and payment APIs for in-app transactions.

Challenge — During peak hours in cities like Ahmedabad and Bengaluru, a single ride-request triggers calls to several external APIs simultaneously. A timeout from any one service could freeze the booking flow for the customer.

What they did — Ola's backend wraps all external API calls with strict timeouts and circuit breakers — if an external service fails, the system falls back to cached data or a simplified response rather than blocking the user. Each external call runs with a dedicated timeout and returns a graceful fallback.

Result — Even during outages of third-party map or payment services, Ola's core ride-booking flow continues with degraded but functional responses, maintaining user experience through peak hours.

Takeaway — A single unguarded external API call can block your entire endpoint. Always set timeouts, catch exceptions, and define a fallback response.

Source: Ola Engineering Blog, tech.olacabs.com (2022)

“Your API is only as reliable as its weakest external dependency — unless you build in the fallbacks.”

WHERE TO GO NEXT

Your DRF server now talks to the outside world. The final session brings everything together: SMS OTPs, custom response formatting, API versioning, and live deployment.

Explore deeper — Public APIs list at <https://github.com/public-apis/public-apis> — filter by category (weather, finance, health) for project ideas

Try on your own — Add a `/api/geocode/` endpoint to your Doctor Finder project and use it to convert the clinic address field into lat/lng coordinates — display them in the API response

Connect the dots — Session 06 adds Twilio SMS, custom error formatting, API versioning, and deploys the entire REST API to PythonAnywhere.

Going further — Explore the `requests-cache` library (`pip install requests-cache`) to cache external API responses for a set number of seconds — reduces latency and avoids hitting rate limits during repeated identical queries.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python backend tutor. Explain in simple Hinglish why a mobile app should call its own backend API instead of calling OpenWeatherMap or Google Maps directly from the app. Use a Doctor Finder app example where the app needs the clinic location on a map. Mention API key security and CORS. Keep it under 150 words.

SESSION
06

60 MIN · PRODUCTION & DEPLOYMENT

Advanced Features + Deployment

◆ Learning Objectives

- Send transactional OTP SMS to an Indian mobile number using the Twilio API
- Apply a global custom exception handler so all error responses follow one consistent JSON structure
- Implement URL-based API versioning (v1, v2) without breaking existing clients
- Deploy a Django REST API to PythonAnywhere and test all live endpoints in Postman
- Describe the deployment checklist: DEBUG=False, ALLOWED_HOSTS, collectstatic, migrate

01 Sending OTP SMS via Twilio

Twilio is the world's most widely used SMS API — it powers OTP verification for Swiggy, Urban Company, and thousands of Indian startups. A free Twilio trial account gives you enough credits to send test SMS messages to one verified Indian mobile number. The integration is a single pip install and five lines of Python inside a DRF view.

```

pip install twilio
# api/views.py - SendOTPAPI
import random, os
from twilio.rest import Client
TWILIO_SID = os.environ.get('TWILIO_ACCOUNT_SID', '')
    From twilio.com/console — never hardcode
TWILIO_TOKEN = os.environ.get('TWILIO_AUTH_TOKEN', '')
    From twilio.com/console
TWILIO_FROM = os.environ.get('TWILIO_PHONE_NUMBER', '')
    The Twilio phone number assigned to your account

class SendOTPAPI(APIView):
    permission_classes = [AllowAny]
    def post(self, request):
        phone = request.data.get('phone', '')
        if not phone or len(phone) != 10:
            return Response({'error': 'Provide a valid 10-digit mobile number'},
                status=status.HTTP_400_BAD_REQUEST)
        otp = ''.join([str(random.randint(0, 9)) for _ in range(6)])
        client = Client(TWILIO_SID, TWILIO_TOKEN)
        client.messages.create(body=f'Your TOPS OTP is: {otp}. Valid for 5 minutes.',
            from_=TWILIO_FROM, to=f'+91{phone}')

```

Install the Twilio SDK — add to requirements.txt

Generates and sends a 6-digit OTP via SMS

random for OTP generation, os for env vars

Twilio Python SDK

POST with a phone number to receive an OTP

Anyone can request an OTP (rate-limit in production)

Handles the OTP send request

Expect 10-digit Indian mobile number, e.g. 9876543210

Basic validation before calling Twilio

Early return on bad input

Generate random 6-digit OTP — store in cache/session in production

Wrap Twilio call in try/except

Create the Twilio client

Send SMS — +91 is India's country code

```
return Response({'status': 'OTP sent', 'otp_preview': otp})
```

In production, remove otp_preview — never expose OTP in the response

```
except Exception as e:                                Twilio auth failure, network error, invalid number
return Response({'error': str(e)}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)
```

Return error message to help the developer debug

TRY THIS LIVE

In Postman: POST to /api/send-otp/ with Body -> raw -> JSON: {"phone": "your_10_digit_number"}. In a Twilio trial account, the receiving number must be verified first at [twilio.com/console](https://www.twilio.com/console). Check your phone for the SMS. Then remove the otp_preview field from the response before committing — exposing OTPs in API responses is a security flaw.

02 Custom Response Formatting

Right now, a successful response returns {"id": 1, "name": "Dr. Shah"} while a failed one returns {"phone": ["This field is required."]}]. These have completely different shapes — confusing for any frontend developer consuming your API. A professional API wraps every response in a consistent envelope: a success flag, a data field, and an error field. DRF lets you enforce this globally with a custom exception handler and a base view mixin.

CONCEPT — Standard API Response Envelope

Every response, success or failure, follows the same structure: {"success": true/false, "data": {...} or null, "error": null or {...}}. Frontend developers write one parseResponse() function and never need to inspect status codes manually. This pattern is used by Razorpay, Juspay, and most production Indian fintech APIs.

```
# api/handlers.py — create this file                                Global exception handler for consistent error responses
from rest_framework.views import exception_handler

DRF's default handler — always call this first
def custom_exception_handler(exc, context):                            Called automatically for every unhandled exception in
                                                                    DRF

    response = exception_handler(exc, context)

    Let DRF handle the exception and produce a base response
    if response is not None:                                           Only modify if DRF produced a response (not a 500
                                                                    crash)

    response.data = {'success': False, 'data': None, 'error': response.data}

    Wrap error payload in the standard envelope
    return response                                                    Return the modified response

# settings.py — register the custom handler:
REST_FRAMEWORK = {
    Tell DRF to use our handler instead of the default
    Add inside your existing REST_FRAMEWORK dict
```

```
'EXCEPTION_HANDLER': 'api.handlers.custom_exception_handler',
    Path: app_name.module_name.function_name
}
```

03 API Versioning — Never Break a Live Client

Once your API is live and a mobile app depends on it, changing the response structure can break thousands of user sessions. Versioning solves this: `/api/v1/doctors/` continues working exactly as before, while `/api/v2/doctors/` introduces the new fields or format. Existing clients are unaffected; new clients adopt v2. URL-based versioning is the most visible and easiest to test with Postman.

URL-Based Versioning

Version is in the URL: `/api/v1/doctors/`. Easy to test in Postman and browsers. Explicit and obvious to any developer reading the URL. Recommended for most projects — use this one.

Header-Based Versioning

URL stays the same: `/api/doctors/`. Version is in a request header: `Accept: application/json; version=v2`. Cleaner URLs but harder to test — you cannot just paste the URL in a browser. Used by GitHub's REST API.

```
# settings.py — add versioning config to REST_FRAMEWORK:
    Enable URL-based versioning globally
'DEFAULT_VERSIONING_CLASS': 'rest_framework.versioning.URLPathVersioning',
    Reads the version from the URL path
    'DEFAULT_VERSION': 'v1',
    'ALLOWED_VERSIONS': ['v1', 'v2'],
# project/urls.py — register both versions:
path('api/v1/', include('api.urls')),
path('api/v2/', include('api_v2.urls')),
# In a ViewSet — serve different serializers per version:
    Optional: detect the version at runtime
    def get_serializer_class(self):
        if self.request.version == 'v2':
            return DoctorSerializerV2
        return DoctorSerializer
```

Version to use if none is specified
Any other version string returns 404
Wire both version namespaces
Existing endpoints — no changes to `api/urls.py`
New version with updated serializers or views

Override in `DoctorViewSet`
Check which version the client requested
New serializer with extra fields (e.g. rating, `profile_photo`)
Original serializer for v1 clients

PRO TIP — Start at v1 Even If You Think You Will Not Need v2

Every API that ships without versioning eventually needs to add it — and by then, breaking the URL structure disrupts live clients. Starting at `/api/v1/` from day one costs nothing and gives you the flexibility to introduce v2 cleanly whenever the need arises. Retrofit versioning is far more painful than planning for it upfront.

04 Deploying to PythonAnywhere

PythonAnywhere is the fastest way to get a Django REST API onto a public URL — no server configuration, no firewall rules, and a free tier that supports one web app. It is ideal for client demos, student projects, and API testing. The steps below take the Doctor Finder API from a local machine to a live URL in under 30 minutes.

```
# On your local machine – push code to GitHub first
PythonAnywhere pulls from GitHub
git add . && git commit -m 'Ready for deployment' && git push
Commit all changes including requirements.txt
# On PythonAnywhere – open a Bash console:      Go to pythonanywhere.com -> Consoles -> Bash
git clone https://github.com/yourusername/doctorfinder.git
Clone your repo into PythonAnywhere
cd doctorfinder                                Enter the project directory
mkvirtualenv --python=/usr/bin/python3.10 doctorenv
Create a Python virtual environment
pip install -r requirements.txt                  Install all dependencies from requirements.txt
python manage.py migrate                        Run database migrations
python manage.py collectstatic --noinput        Collect static files for the browsable API and admin
# In PythonAnywhere Web tab:                    Configure the web app manually
# Source code: /home/yourusername/doctorfinder
Point to your project root
# Virtualenv: /home/yourusername/.virtualenvs/doctorenv
Point to the env you just created
# WSGI file: update import_name = 'doctorfinder.wsgi'
Match your project's wsgi.py path
# settings.py – change for production:          Critical before going live
DEBUG = False                                  Never run DEBUG=True in production
ALLOWED_HOSTS = ['yourusername.pythonanywhere.com']
Add your PythonAnywhere subdomain
```

TRY THIS LIVE

After deployment, open Postman and change the base URL from `http://127.0.0.1:8000` to `https://yourusername.pythonanywhere.com`. Run GET `/api/v1/doctors/` — you should see your live data. Share the URL with your trainer and classmates. Test the token endpoint, weather endpoint, and OTP endpoint all from the live URL.

05 Module 5 Wrap-Up — What You Can Now Build

CONCEPT — What a Complete DRF API Looks Like

Across 6 sessions you have built: a versioned REST API (v1/v2) with full CRUD, token authentication, role-based permissions, pagination and search filters, external API integrations (weather, maps, SMS), a consistent error response format, and a live public deployment on PythonAnywhere. This is a production-grade API stack — the same components used by Indian startups from seed stage to Series B.

CASE STUDY — Zepto, Mumbai

Context — Zepto is a Bengaluru/Mumbai-based quick-commerce startup delivering groceries in 10 minutes. Launched in 2021, it reached a \$1.4 billion valuation within 18 months of founding.

Challenge — At launch, Zepto needed a live API for their mobile app within weeks — covering OTP login, product listings, order creation, and delivery tracking — while maintaining backward compatibility as features were added rapidly.

What they did — Zepto's early engineering team used URL-based API versioning from the first release, Twilio for OTP verification, structured error responses for clean frontend integration, and PythonAnywhere-equivalent cloud hosting for their MVP before migrating to AWS as traffic scaled.

Result — The versioned API allowed the mobile team to ship app updates independently of backend changes. OTP-verified login reduced fraudulent orders. Clean error formatting cut frontend debugging time significantly during the high-speed early months.

Takeaway — The features covered in this session — versioning, OTP, consistent errors, and deployability — are not advanced extras. They are what makes a student project production-ready.

Source: Zepto Engineering Blog and YourStory founding story coverage (2022)

“A local API is a proof of concept. A deployed, versioned, authenticated API is a product.”

WHERE TO GO NEXT

You have completed the Django REST Framework module. The next step is applying this API layer to the Doctor Finder capstone from Module 4 — replacing its Django HTML views with DRF endpoints and connecting it to a React or mobile frontend.

Explore deeper — DRF official API versioning guide at <https://django-rest-framework.org/api-guide/versioning/>

Try on your own — Add versioning to your Doctor Finder API: create a `DoctorSerializerV2` that adds an `average_rating` field, wire it under `/api/v2/doctors/`, and confirm `/api/v1/doctors/` still returns the original format

Connect the dots — The full capstone project combines Module 4 (Doctor Finder Django app) + Module 5 (DRF API layer) into a deployable full-stack backend — the Doctor Finder API available live at your PythonAnywhere URL.

Going further — Explore AWS Elastic Beanstalk or Railway.app as production-grade alternatives to PythonAnywhere — both support auto-scaling, custom domains, and environment variable management for when your API outgrows a single server.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Django backend deployment tutor. Explain in simple Hinglish the steps to deploy a Django REST Framework API on PythonAnywhere. Include `DEBUG=False`, `ALLOWED_HOSTS`, `collectstatic`, and `migrate` in your explanation. Use an analogy of moving a shop from your home to a proper market. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 5

Interview Questions

◆ Django REST Framework (REST API Development)

Rehearse each answer out loud and aim to deliver it within 60 seconds. The hint is only a nudge — attempt the answer first, then compare with the model answer below.

1

What is an API and why are APIs important in web development?

Think about how different systems communicate with each other.

MODEL ANSWER

An API (Application Programming Interface) is a contract that defines how two software systems exchange data. In web development, APIs let a frontend app, a mobile app, and third-party services all talk to the same backend without knowing its internal implementation. They enable modularity, team independence, and the ability to expose your service to any client platform.

2

Explain what a REST API is and name two key principles it follows.

Consider statelessness and how resources are handled.

MODEL ANSWER

REST (Representational State Transfer) is an architectural style for building APIs over HTTP. Two key principles are statelessness — each request must contain all the information needed; the server stores no client context between requests — and resource-based URLs, where each URL represents a resource (e.g., `/api/doctors/`) and HTTP methods (GET, POST, PUT, DELETE) define the action.

3

What is the difference between JSON and XML when it comes to data exchange in APIs?

Focus on readability and typical use in modern APIs.

MODEL ANSWER

JSON (JavaScript Object Notation) uses a lightweight key-value structure that is easy to read and directly parseable in JavaScript. XML uses verbose opening and closing tags, making it heavier and harder to read. Modern REST APIs almost universally use JSON because of its simplicity, smaller payload size, and native support in virtually every programming language.

4

How is Django REST Framework (DRF) different from standard Django?

Think about what DRF adds to Django for API development.

MODEL ANSWER

Standard Django is built for server-rendered HTML responses. DRF extends Django to produce and consume JSON (or other formats), adding serializers, API views, authentication classes, permissions, throttling, and a browsable API interface. DRF handles content negotiation, request parsing, and response rendering that you would otherwise have to write manually.

5

Describe the steps to install Django REST Framework in a Django project.

Consider both package installation and settings configuration.

MODEL ANSWER

Run `pip install djangorestframework`. Then add `'rest_framework'` to `INSTALLED_APPS` in `settings.py`. Optionally add a `REST_FRAMEWORK` dictionary in settings for global configuration such as default authentication and permission classes. That is all — DRF is now available for use in your views and serializers.

6

What is the purpose of adding 'rest_framework' to INSTALLED_APPS in Django settings?

MODEL ANSWER

Registering `'rest_framework'` in `INSTALLED_APPS` tells Django to load DRF's static files (for the browsable API), management commands, and template tags. Without it, the browsable API interface will not render and DRF's built-in views may not function correctly. It follows Django's app registration convention for all installed packages.

7

What is the difference between a Serializer and a ModelSerializer in DRF?

Think about which one is more automatic and when you'd use each.

MODEL ANSWER

A Serializer requires you to manually define every field and write `create()` and `update()` methods yourself. A ModelSerializer auto-generates fields from a model specified in its Meta class and provides default `create()` and `update()` implementations. Use ModelSerializer when your API maps closely to a model, and a plain Serializer for complex or non-model data transformations.

8

How would you create a simple 'Hello API' endpoint in DRF using APIView?

MODEL ANSWER

```
from rest_framework.views import APIView
from rest_framework.response import Response
```

```
class HelloAPI(APIView):
    def get(self, request):
        return Response({'message': 'Hello, World!'})
```

Map it in `urls.py`: `path('hello/', HelloAPI.as_view())`. APIView handles content negotiation and provides `request.data` for parsed input.

9

What is a Django model, and why do we define models before creating API endpoints?

Consider how models relate to the database.

MODEL ANSWER

A model is a Python class that maps to a database table and defines the data structure. We define models first because serializers (especially ModelSerializers) derive their fields from the model, and API views query the model to retrieve or save data. Without a model there is no persistent data for the API to serve.

10**Write a basic DRF Serializer for a model called Doctor with fields 'name' and 'specialization'.***Remember to import Serializer and define fields.***MODEL ANSWER**

```
from rest_framework import serializers
from .models import Doctor
```

```
class DoctorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Doctor
        fields = ['id', 'name', 'specialization']
```

This is all that is needed for basic serialisation. DRF automatically maps model field types to serializer field types.

11**How do you create a CRUD API endpoint for a model using APIView in DRF?***Think about the HTTP methods involved and their mapping.***MODEL ANSWER**

Create a class-based APIView with methods matching HTTP verbs: get() for listing or retrieving, post() for creating, put() for full update, patch() for partial update, and delete() for deletion. Each method uses the serializer to validate and transform data and returns a Response with the appropriate status code. Map the view to a URL in urls.py.

12**What is the difference between HTTP status codes 200 and 201 in API responses?***One is for successful retrieval, the other for successful creation.***MODEL ANSWER**

200 OK means the request was successful and the response body contains the requested data — used for GET and successful PUT/PATCH. 201 Created means a new resource was successfully created — used after a successful POST. Returning 201 instead of 200 on creation is important for API clients to distinguish a retrieval from an insertion.

13**How would you return a JSON response from a DRF API endpoint?****MODEL ANSWER**

Use DRF's Response class: from rest_framework.response import Response; return Response({'key': 'value'}, status=200). DRF's Response handles content negotiation automatically — it returns JSON by default. Unlike Django's JsonResponse, it works with the full DRF content negotiation pipeline and respects Accept headers from the client.

14

Write the code for a POST endpoint to create a new Doctor using ModelSerializer in DRF.

MODEL ANSWER

```
class DoctorListCreate(APIView):
    def post(self, request):
        serializer = DoctorSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
        return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)
```

`is_valid()` validates the incoming data against model constraints. `save()` calls the serializer's `create()` method. Returning errors with 400 gives the client actionable feedback.

15

What is the use of GenericAPIView and Mixins in DRF?

Think about reducing code repetition for common operations.

MODEL ANSWER

GenericAPIView extends APIView with a `queryset` and `serializer_class` attribute, providing common behaviour like `get_queryset()` and `get_object()`. Mixins (`ListModelMixin`, `CreateModelMixin`, etc.) add specific methods like `list()` and `create()`. Combining them — class `DoctorList(ListModelMixin, GenericAPIView)` — gives you standard CRUD operations with very little code.

16

Describe how you would update a Doctor object using a PUT request in DRF.

MODEL ANSWER

```
def put(self, request, pk):
    doctor = get_object_or_404(Doctor, pk=pk)
    serializer = DoctorSerializer(doctor, data=request.data)
    if serializer.is_valid():
        serializer.save()
    return Response(serializer.data)
    return Response(serializer.errors, status=400)
```

Passing the existing instance as the first argument to the serializer tells it to update rather than create. All fields must be provided with PUT; use `partial=True` for PATCH.

17

Explain what a ViewSet is in DRF and how it differs from APIView.

Focus on how ViewSets group logic and reduce code.

MODEL ANSWER

A ViewSet groups the logic for a related set of views — list, create, retrieve, update, destroy — into a single class. Instead of defining separate classes for each action, you define them as methods (`list()`, `create()`, etc.) in one ViewSet. Unlike APIView, ViewSets work with Routers that auto-generate URLs, reducing boilerplate significantly.

18

What is the purpose of a Router in DRF, and what does it generate automatically?

Think about URL patterns for your API.

MODEL ANSWER

A Router automatically generates URL patterns for a ViewSet. Registering a ViewSet with `router.register('doctors', DoctorViewSet)` creates `/api/doctors/` (for list and create) and `/api/doctors/{pk}/` (for retrieve, update, and delete). This eliminates the need to manually write `path()` entries for each action, keeping `urls.py` clean.

19

Explain the difference between DefaultRouter and SimpleRouter in DRF.

MODEL ANSWER

Both auto-generate URL patterns for ViewSets. SimpleRouter creates standard URLs for list and detail actions. DefaultRouter does everything SimpleRouter does but additionally creates an API root endpoint at `/` that lists all registered routes, making the API browsable and discoverable. Use DefaultRouter in development for the self-documentation benefit; SimpleRouter is fine in production if you do not need the root view.

20

How does pagination help when returning large querysets from an API?

Consider both server performance and client usability.

MODEL ANSWER

Without pagination, a request for all doctors could return thousands of rows, causing slow queries, large response payloads, and poor frontend rendering. Pagination splits results into pages — returning, say, 20 records with metadata about total count and next/previous page URLs. This keeps response times fast, reduces bandwidth, and gives clients a manageable amount of data per request.

21

Describe how you would implement PageNumberPagination for a list endpoint in DRF.

MODEL ANSWER

In `settings.py` set `REST_FRAMEWORK = {'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination', 'PAGE_SIZE': 10}`. DRF automatically paginates any list view that returns a queryset. Alternatively, create a custom class: `class MyPagination(PageNumberPagination): page_size = 10` and assign it to `pagination_class` on the view. The response will include `count`, `next`, `previous`, and `results` keys.

22

What is the difference between LimitOffsetPagination and CursorPagination in DRF?

MODEL ANSWER

LimitOffsetPagination accepts `limit` and `offset` query parameters — the client requests 'give me 10 records starting at position 30'. It is intuitive but allows clients to jump to any page. CursorPagination uses an opaque cursor (an encoded position) in the URL instead of a page number, making it tamper-proof and efficient for large datasets where the data changes frequently, such as a live feed.

23

How would you add ordering by 'name' to a list endpoint in DRF?**MODEL ANSWER**

Add `OrderingFilter` to `filter_backends` on the view and set `ordering_fields = ['name', 'specialization']`. Clients can then pass `?ordering=name` for ascending or `?ordering=-name` for descending. Alternatively set `ordering = ['name']` on the view as the default order. The filter handles everything — no custom queryset manipulation needed.

24

What are the main types of authentication available in DRF?*List at least three and describe them briefly.***MODEL ANSWER**

`BasicAuthentication` sends username and password with every request in a Base64-encoded header — simple but insecure without HTTPS. `SessionAuthentication` uses Django's session cookies, suitable for browser-based clients. `TokenAuthentication` issues each user a token; clients include it in the Authorization header. `JWTAuthentication` (via `django-rest-framework-simplejwt`) uses signed JSON tokens that contain user claims and expire automatically.

25

Explain the difference between `TokenAuthentication` and `SessionAuthentication` in DRF.*Consider where the authentication information is stored.***MODEL ANSWER**

`TokenAuthentication` stores a token in the database and the client sends it in the Authorization header on every request — stateless from the server's perspective and suitable for mobile apps and third-party clients. `SessionAuthentication` relies on a server-side session and a browser cookie, making it suitable for same-origin browser clients but not for mobile or cross-origin API consumers.

26

Write a DRF permission class that only allows access to authenticated users.**MODEL ANSWER**

```
from rest_framework.permissions import IsAuthenticated
```

```
class MyView(APIView):  
    permission_classes = [IsAuthenticated]
```

```
def get(self, request):  
    return Response({'data': 'secure'})
```

`IsAuthenticated` is a built-in DRF permission. If the request has no valid credentials, DRF returns a 401 Unauthorized response automatically.

27

How would you protect the `/api/doctors/` endpoint so that only logged-in users can access it?

MODEL ANSWER

Set `permission_classes = [IsAuthenticated]` on the view or globally in `REST_FRAMEWORK` settings as `DEFAULT_PERMISSION_CLASSES`. Pair it with an authentication class such as `TokenAuthentication`. Unauthenticated requests receive a 401 response. For fine-grained control, write a custom permission class that checks specific user attributes.

28

What is the role of the `IsAdminUser` permission class in DRF?

MODEL ANSWER

`IsAdminUser` restricts access to users whose `is_staff` attribute is `True` — typically Django superusers and staff accounts. Any non-staff request is rejected with a 403 Forbidden response. It is used to protect admin-only endpoints such as bulk operations, user management, or analytics without building a full custom permission.

29

If you wanted to create a custom permission that only allows doctors with a certain specialization to edit their profile, how would you approach this in DRF?

Think about overriding the `'has_permission'` or `'has_object_permission'` methods.

MODEL ANSWER

Create a class inheriting `BasePermission` and override `has_object_permission(self, request, view, obj)`. For safe GET methods return `True`; for PUT/PATCH/DELETE check that `request.user == obj.user` and that the user's specialization matches the required value. Return `True` to allow or `False` to deny. Set this as `permission_classes` on the view.

30

How would you generate and use an authentication token for a user in DRF?

MODEL ANSWER

Ensure `'rest_framework.authtoken'` is in `INSTALLED_APPS` and run migrate. Then call `Token.objects.get_or_create(user=user)` in your login view and return the token key in the response. The client stores this token and sends it as `Authorization: Token <key>` in subsequent requests. DRF's `TokenAuthentication` extracts and validates it automatically.

31

What is JWT and how is it different from basic token authentication?

Think about the structure and usage of JWTs.

MODEL ANSWER

JWT (JSON Web Token) is a self-contained token with three Base64-encoded parts: header, payload (claims like user ID and expiry), and signature. Unlike DRF's basic Token (an opaque string looked up in the database on every request), JWT is stateless — the server verifies the signature without a database lookup. JWTs also carry an expiry and can include custom claims, making them popular for scalable APIs.

32

How could you use ChatGPT or Copilot to help you explain the difference between Token and Session Authentication in an interview? What would you check before trusting the output?

Think about verifying explanations and cross-checking with official docs.

MODEL ANSWER

Ask ChatGPT to give a plain-English comparison of the two, then use that as a starting point for your own answer. Before trusting it, cross-check the key claims against the official DRF documentation, since AI tools sometimes conflate details or use slightly outdated terminology. Practise saying the explanation out loud until it is in your own words — interviews reward understanding, not memorised text.

33

Describe the steps to integrate an external API, such as OpenWeatherMap, into a DRF endpoint.

MODEL ANSWER

In your DRF view, use the requests library to call the external API URL with your API key. Parse the JSON response and extract the data you need. Return the relevant fields in a DRF Response object. Store the API key in environment variables or Django settings — never hard-code it. Add error handling for network failures and non-200 responses from the external API.

34

Write a DRF view that calls an external weather API and returns the temperature for a given city.

MODEL ANSWER

```
import requests
from rest_framework.views import APIView
from rest_framework.response import Response

class WeatherView(APIView):
    def get(self, request, city):
        url = f'https://api.openweathermap.org/data/2.5/weather?q={city}&appid=YOUR_KEY&units=metric'
        r = requests.get(url)
        data = r.json()
        temp = data['main']['temp']
        return Response({'city': city, 'temperature': temp})
```

Always add try-except around the external call to handle connection errors gracefully.

35

How would you handle errors if the external API you called from your DRF endpoint is down or returns invalid data?

Consider exception handling and response status codes.

MODEL ANSWER

Wrap the external call in a try-except block catching requests.exceptions.RequestException for network errors. Check the response status code — if it is not 200, return an appropriate error Response. Use .get() with defaults when accessing JSON keys to avoid KeyError if the response structure is unexpected. Log the error details server-side and return a user-friendly 503 Service Unavailable or 502 Bad Gateway to the client.

36

What is Postman and how can it help you test your DRF API endpoints?*Focus on features that make API testing easier.***MODEL ANSWER**

Postman is a GUI tool for sending HTTP requests to APIs without writing any client code. You can set the URL, method, headers (including Authorization tokens), and request body, then inspect the full response including status code, headers, and JSON body. It supports saving requests in collections, running automated test scripts, and sharing API collections with your team — making it the standard tool for API development and testing.

37

How could you use Postman AI or ChatGPT to generate code snippets for calling your DRF API from Python?*Think about the 'Code' feature and checking if the code matches your API's requirements.***MODEL ANSWER**

In Postman, after configuring a request, click 'Code' and select Python (requests) to generate a ready-to-run snippet. Alternatively, describe your endpoint to ChatGPT — method, URL, headers, and body — and ask for a Python requests example. Before using either, verify the Authorization header format matches your DRF setup, check that query parameters are correct, and test the snippet against your actual running API.

38

Explain how you would send an email using an external service like Mailgun or MailChimp from a DRF endpoint.**MODEL ANSWER**

Configure Django's email backend to use the service's SMTP credentials in settings.py. In your DRF view, call Django's `send_mail()` after a triggering action — for example, after a successful booking POST. For Mailgun, you can also use their HTTP API directly with the requests library, passing your domain and API key. Store all credentials in environment variables and never hard-code them in source files.

39

Describe how you would integrate Twilio to send an SMS notification from your DRF API.**MODEL ANSWER**

Install the twilio library and load `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, and `TWILIO_PHONE` from environment variables. In the DRF view, after the relevant action is completed, instantiate the Twilio Client and call `client.messages.create(body='Your appointment is confirmed', from_=TWILIO_PHONE, to=user_phone)`. Handle `twilio.base.exceptions.TwilioRestException` to catch API errors and log them without breaking the main response.

40

What steps are involved in deploying a DRF API to PythonAnywhere?*Think about preparing your project, setting up the web app, and configuring settings.***MODEL ANSWER**

Push your project to GitHub and clone it on PythonAnywhere via Bash. Create a virtualenv and install dependencies from requirements.txt. Create a Web App (manual config), point the WSGI file to your Django project. Set `DEBUG=False`, configure `ALLOWED_HOSTS`, set up the database (run migrations), and configure static files. Add your secret key and API credentials as environment variables. Reload the web app and test the API endpoints.

41

Why is API versioning important, and how would you implement it in DRF?

Consider backward compatibility and URL structure.

MODEL ANSWER

Versioning lets you introduce breaking changes to your API without disrupting existing clients that rely on the old behaviour. The simplest approach is URL versioning — prefix all routes with `/api/v1/` or `/api/v2/`. In DRF, set `DEFAULT_VERSIONING_CLASS` to `URLPathVersioning` and access the version in views via `request.version`. Each version can use different serialisers or views while sharing the same models.

42

How would you format a custom JSON response in DRF, for example, to wrap all data in a 'result' key?

MODEL ANSWER

Override DRF's `Response` by creating a custom renderer or a utility function. A simple approach is a helper:

```
def api_response(data, status=200):
    return Response({'result': data, 'status': 'success'}, status=status)
```

Call this in every view instead of returning `Response` directly. For a project-wide solution, write a custom pagination class or a custom `Response` subclass that wraps data automatically.

43

Describe how exception handling in DRF can improve the robustness of your API.

MODEL ANSWER

DRF's exception handler converts unhandled exceptions into structured JSON responses rather than HTML error pages. You can customise it by setting `EXCEPTION_HANDLER` in `REST_FRAMEWORK` settings to your own function. This lets you return consistent error shapes — always including an error code and message — regardless of where the exception occurred, making your API easier for clients to handle programmatically.

44

Imagine you have deployed your DRF API and a client reports that the `/api/doctors/` endpoint returns a 500 error. How would you troubleshoot this?

Think about checking logs, error messages, and recent code changes.

MODEL ANSWER

First check the server error logs on PythonAnywhere or your hosting platform for the traceback. Reproduce the request locally with the same data using Postman or curl. Check recent code changes in version control for the likely culprit. Common causes include missing database migrations, a missing environment variable, a serialiser field mismatch, or a broken external API call. Fix the root cause, run tests, and redeploy.

45

Integrative: How would you secure an endpoint that integrates with an external payment API, ensuring only authenticated users can access it and handling errors gracefully?

Combine authentication, permissions, and exception handling.

MODEL ANSWER

Set `permission_classes = [IsAuthenticated]` and `authentication_classes = [TokenAuthentication]` on the payment view. Wrap the external payment API call in a `try-except` to catch network errors and API failures, returning a 502 with a clear error message if the gateway is unreachable. Verify the payment response signature before updating the database. Log all payment attempts with user ID, amount, and outcome for audit purposes.

46

Integrative: Describe how you would create a paginated, authenticated endpoint that lists doctors and allows filtering by specialization.

MODEL ANSWER

Use a `ListAPIView` with `permission_classes = [IsAuthenticated]`, `serializer_class = DoctorSerializer`, and `queryset = Doctor.objects.all()`. Set `pagination_class` to `PageNumberPagination` with `page_size = 10`. Add `DjangoFilterBackend` to `filter_backends` and set `filterset_fields = ['specialization']`. A request like `GET /api/doctors/?specialization=Cardiology&page=2` returns the second page of cardiologists.

47

Integrative: Suppose you need to update your API to v2, adding a new required field to the Doctor model. How would you handle this change to avoid breaking existing clients?

Think about versioning and backward compatibility.

MODEL ANSWER

First add the new field to the model with a default value or `null=True` so the migration does not break existing records. Create a v2 serializer that includes the new field as required. Set up URL versioning so `/api/v1/doctors/` continues to use the old serializer while `/api/v2/doctors/` uses the new one. Communicate the deprecation timeline to v1 clients and monitor usage before eventually retiring v1.